

Capstone Project

Two-Wheeler Post-Crash Detection System

ML Enabled Embedded Classification with False-Positive
Suppression



Problem Statement

01 / THE CRISIS

- ~1.5 lakh road accident deaths annually in India
- 44.5% of road fatalities involve two-wheelers
- Riders have minimal physical protection during crashes
- Post-crash response time is critical for survival
- Unconscious riders may be unable to call for help

02 / THE GAP

- Smartphone apps: unreliable after impact and dependent on the phone
- OEM telematics: typically ₹5,000–₹25,000 and difficult to retrofit
- Low-cost systems: often rely on simple acceleration thresholds
- Indian roads create false triggers: potholes, speed breakers and hard braking

03 / THE SOLUTION

- Sub-₹2,000 retrofit device for existing two-wheelers
- Multi-sensor fusion: acceleration + gyroscope + GPS
- Classifies Crash / Pothole / Hard Braking / Normal
- Uses false-positive-aware decision logic instead of a single threshold
- 30-second cancellation window before emergency alert
- Sends SMS + GPS location to emergency contacts

Project Overview & Key Features

Sensors

Detection

Classification

GPS

Alert

Human
Confirmation

MULTI-SENSOR CORE

6-axis IMU crash detection • GPS at impact • SMS + BLE notification

FALSE-POSITIVE AWARE

Suppresses pothole, speed breaker, kickstand drop and hard-braking events

HUMAN-IN-LOOP

30-second cancellation window before emergency escalation

DESIGN INTENT

Retrofittable, India-specific and operational below ₹2,000

Action Plan: Implementation Timeline

Phase 01 / HARDWARE + DATA

- Source ESP32, MPU6050, GPS; verify protocols
- Integrate SIM800L; test SMS + AT commands
- Mount prototype; 50+ labeled events/class

Phase 02 / FIRMWARE + CLASSIFIER

- Analyze data; validate thresholds
- C++ rules + 30s cancel window
- 50+ event stress test; FP/FN measurement

Phase 03 / INTEGRATION

- App + weatherproof enclosure
- Demo videos + controlled drops
- Final report + Paper draft

Action Plan & Timeline

Phase 1

- Hardware assembly
- Data collection
- Component validation
- Model Training

Phase 2

- Firmware development
- Implementation & Testing Model
- Stress testing
- Iterate

Phase 3

- Companion app (If needed)
- Weather enclosure
- Testing
- Iterate
- Report + Prototype

Working Hardware Prototype + Web App

Bill of Materials

CORE SENSORS + PROCESSING

- ESP32 DevKit v1 ₹350–400
- MPU6050 ₹80–120
- NEO-6M GPS ₹300–400

POWER + COMMUNICATION

- SIM800L ₹350–450
- 18650 Li-Ion ₹150–200
- MT3608 ₹20–40
- TP4056 ₹30–50

INTERFACE + ENCLOSURE

- Tactile button ₹5–10
- Small buzzer ₹20–30
- ABS box + jumpers ₹130–230

TARGET COST < ₹2,000 INR

Component Specifications & Rationale

MPU6050 / 6-AXIS IMU

Proven low-cost accelerometer + gyroscope.
I²C interface; integrated gyro eliminates a separate sensor.

NEO-6M / GPS

Cost-effective, industry-standard positioning module for IoT and vehicle location tagging.

SIM800L / GSM

2G GSM: low-cost and reliable in rural India. SMS-only communication—no mobile data required.

ESP32 / CONTROL

Dual-core MCU with WiFi + Bluetooth. 520KB SRAM fits firmware, buffering and decision logic.

Data Flow: Raw Sensors → Classification

RAW INPUT

- MPU6050 IMU captures motion data
- 100 Hz sampling
- Ax, Ay, Az → 3-axis acceleration
- Rx, Ry, Rz → 3-axis gyroscope
- Continuous data collection enables real-time event detection

SIGNAL PROCESSING

- Moving average filter ($w = 5$) reduces sensor noise
- Estimate post-event tilt using accelerometer data
- Prepare clean signals for reliable feature extraction

FEATURES

- Peak-G
- Jerk
- Post-Tilt
- Stillness Duration
- Event Duration

CLASSIFICATION

Four target classes:

- CRASH
- POTHOLE
- HARD BRAKING
- NORMAL

ALERT

Buzzer
SMS + BLE
30s cancellation timer

- Crash detected → local buzzer activates
- Start a 30-second cancellation window

How to Interlink with ML Model

ML Model Details

- Decision Tree trained on labeled sensor data
- Inputs: Peak-G, Jerk, Tilt, Stillness, Duration
- Classes: Crash / Pothole / Braking / Normal
- Lightweight model for ESP32 inference

TINYML EXTENSION to ESP32

- Train model on Laptop/Python
- Convert learned model to C/C++ / TinyML
- Deploy directly on ESP32
- Real-time, offline crash classification

Test - Repeat

- Test using collected riding data
- Compare accuracy & false-positive rate
- Confusion matrix + precision/recall
- Optimize model for low latency & low FP

FROM SIMULATION TO ESP32 HARDWARE

Deployment Strategy

Simulation

—

Hardware

—

Firmware

—

Testing

—

Optimization

01 / SETUP + VALIDATION

Assemble components and verify I²C, UART, GPIO and power paths.

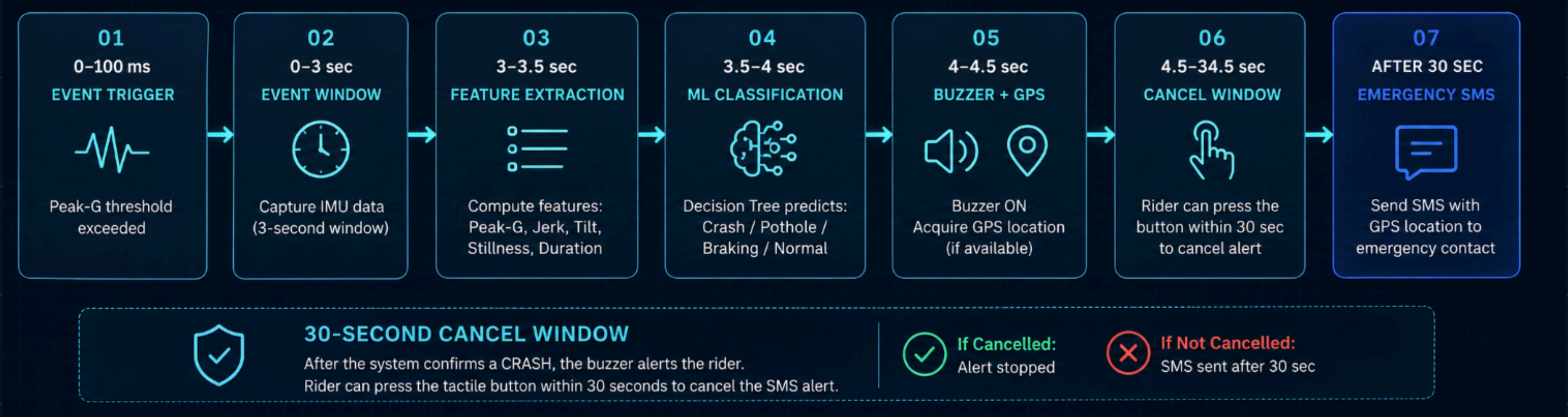
02 / C++ FIRMWARE

Implement event detection, classifier, alert logic and cancellation state.

03 / TEST + ITERATE

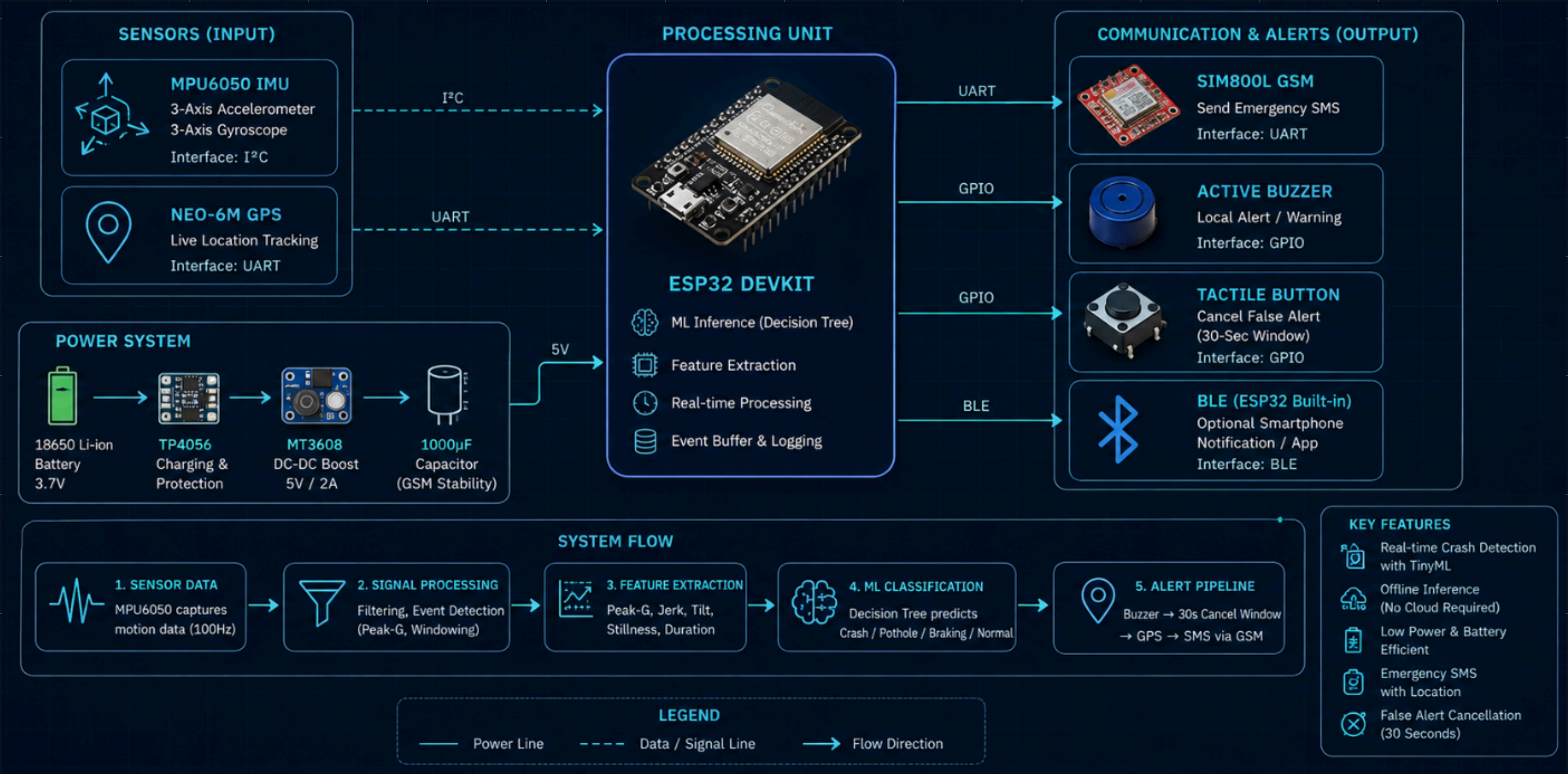
Real events • latency <2s • target false-positive rate <5%

End-to-End Crash Alert Pipeline



After CRASH_CONFIRMED, the rider can press the tactile button to cancel the SMS alert.

Architecture / Blueprint



Evaluation Methodology & Key Metrics

DATASET COLLECTION

- 50+ events per class
- Classes: Crash, Pothole, Hard Braking, Normal
- Collect using MPU6050 + ESP32
- Manual labeling with timestamp + event type
- Use controlled tests for crash/drop events
- Store sensor data as CSV

PERFORMANCE METRICS

- Accuracy
- Precision & Recall
- F1 Score
- Confusion Matrix
- False-positive rate
- Detection latency
- Target: <2 sec classification latency

VALIDATION STRATEGY

- 80% training / 20% testing
- Keep test data separate from training
- Repeat controlled tests with different conditions
- Test real-world potholes, braking & normal riding

Two-Wheeler Crash Detection: A System for Real-World Impact

Embedded Classification | India-Specific Validation | Open-Source Research

“Detect the impact. Suppress the noise. Save the seconds that matter.”

Thank You

